FIND MIDDLE ELEMENT IN THE LIST

```
#include <stdio.h>
#include <stdlib.h>
// Node structure
struct Node {
    int data;
    struct Node* next;
};
// Function to find middle element
void findMiddle(struct Node* head) {
    if (head == NULL) {
        printf("List is empty\n");
        return;
    }
    struct Node* slow = head;
    struct Node* fast = head;
    while (fast != NULL && fast->next != NULL) {
        slow = slow->next;
        fast = fast->next->next;
    }
    printf("Middle element: %d\n", slow->data);
}
```

```
// Display list
void display(struct Node* head) {
    struct Node* temp = head;
    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}
int main() {
    int n, i, value;
    struct Node *head = NULL, *temp = NULL, *newNode =
NULL;
    printf("Enter number of nodes: ");
    scanf("%d", &n);
    if (n <= 0) {
        printf("Invalid number of nodes\n");
        return 0;
    }
```

```
    // Create list using user input
    for (i = 0; i < n; i++) {
        printf("Enter value for node %d: ", i + 1);
        scanf("%d", &value);
        newNode = (struct Node*)malloc(sizeof(struct
Node));
        newNode->data = value;
        newNode->next = NULL;

        if (head == NULL) {
            head = newNode;
            temp = newNode;
        } else {
            temp->next = newNode;
            temp = newNode;
        }
    }
    printf("Linked List: ");
    display(head);
    findMiddle(head);
    return 0;
}
```

while (fast != NULL && fast->next != NULL)

WHY AND OPERATOR WHY NOT OR OPERATOR

The condition:

```
while (fast != NULL && fast->next != NULL)
```

Why we use `&&` (AND)

We are doing:

```
fast = fast->next->next;
```

So before moving `fast`, we must ensure:

1. `fast` is not NULL
2. `fast->next` is not NULL

Only then is `fast->next->next` safe to access.

That's exactly what `&&` ensures — **both conditions must be true**.

What happens if you use `||` (OR)?

If you write:

```
while (fast != NULL || fast->next != NULL)
```

This is dangerous ✕

Case 1:

If `fast == NULL`, then:

```
fast->next
```

will be accessed → □ **Segmentation fault (crash)**

Because:

- `||` allows loop to continue even if one condition is false
- So it might still evaluate `fast->next` when `fast` is NULL

Key Concept: Short-Circuit Evaluation

In C:

- `&&` stops if first condition is false → safe
- `||` stops if first condition is true → unsafe in this case